

SESSION 10

Forgot Password + Session Management

Topics Covered: Django sessions, Reset password with OTP link, Session expiry, Middleware basics

Activities: Implement forgot password, Store OTP in session

Task 1: Create Django view `forgot_password_view` rendering form where users enter email to request password reset.

Explanation & Approach: *Renders initial entry page for single-use OTP password restoration workflow.*

```
# views.py
from django.shortcuts import render

def forgot_password_view(request):
    return render(request, 'accounts/forgot_password.html')
```

Task 2: Generate random 6-digit OTP, store in session, send to user email using `send_mail` function.

Explanation & Approach: *`request.session['otp']` stores transient OTP codes attached to specific user session cookies.*

```
# views.py
import random
from django.shortcuts import render, redirect
from django.core.mail import send_mail

def send_otp_view(request):
    if request.method == 'POST':
        email = request.POST['email']
        otp = str(random.randint(100000, 999999))

        # Save OTP in session
        request.session['otp'] = otp
        request.session['reset_email'] = email
        request.session.set_expiry(300) # 5 minutes expiry

        # Send Email
        send_mail(
            'Your Password Reset OTP',
            f'Your 6-digit OTP for password reset is: {otp}. Valid for 5 minutes.',
            'auth@app.com',
            [email]
        )
        return redirect('verify_otp')
    return render(request, 'accounts/forgot_password.html')
```

Task 3: Build Django form where user enters OTP received via email, validate by comparing with session value.

Explanation & Approach: Comparing user POST data against `request.session['otp']` ensures secure identity verification.

```
# views.py
from django.shortcuts import render, redirect
from django.contrib import messages

def verify_otp_view(request):
    if request.method == 'POST':
        user_otp = request.POST['otp']
        session_otp = request.session.get('otp')

        if session_otp and user_otp == session_otp:
            messages.success(request, "OTP Verified! Enter new password.")
            return redirect('reset_password_page')
        else:
            messages.error(request, "Invalid or expired OTP. Please try again.")
            return render(request, 'accounts/verify_otp.html')

    return render(request, 'accounts/verify_otp.html')
```

Task 4: Set session expiry time of 5 minutes for OTP so it becomes invalid after that period.

Explanation & Approach: `request.session.set_expiry(300)` automatically deletes session keys after 300 seconds.

```
# Setting session timeout explicitly
request.session['otp'] = otp
request.session.set_expiry(300) # 300 seconds = 5 minutes
```

Task 5: Create custom middleware `BlockExpiredOTPAccessMiddleware` checking if OTP session key is missing/expired, redirecting user to forgot password form.

Explanation & Approach: Custom Django middleware intercepts incoming requests to enforce session lifecycle rules globally.

```
# middleware.py
from django.shortcuts import redirect

class BlockExpiredOTPAccessMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        # Intercept access to OTP verification page
        if request.path == '/accounts/verify-otp/':
```

```
        if 'otp' not in request.session:  
            return redirect('forgot_password') # Redirect if expired or missing  
  
    response = self.get_response(request)  
    return response
```